

Jelinski Pedology Group Data Registry Handbook

Nic Jelinski

2026-09-14

Table of contents

1	Lab Data Registry	8
1.1	What the registry contains	8
1.2	Start here	9
1.3	Core idea in one sentence	9
1.4	Documentation scope	9
2	Overview	10
2.1	Why a dataset registry?	10
2.2	What is being registered?	10
2.3	What the registry is not	11
2.4	Registry vs. scientific schema	11
2.5	Registry vs. glossary	11
2.6	Intended users	12
3	Design principles	13
3.1	Machine-readable first	13
3.2	Dataset identity is distinct from release identity	13
3.3	Raw acquisitions are immutable	14
3.4	Integrity is explicit	14
3.5	Identifiers are stable	14
3.6	People and organizations remain visible	15
3.7	Controlled values are explicit	15
3.8	Uncertainty is documented, not hidden	15
3.9	Automation assists curation; it does not replace review	15
3.10	The system should fail visibly	16
4	Repository architecture	17
4.1	Recommended project tree	17
4.2	registry/	18
4.2.1	registry--packages.csv	18
4.2.2	registry--parties.csv	18
4.2.3	registry--party-roles.csv	18
4.2.4	registry--dictionary.csv	18
4.2.5	registry--vocab-schema.csv	19
4.3	vocab/	19

4.4	<code>data/</code>	19
4.5	<code>scripts/</code>	20
4.6	<code>docs/</code>	20
4.7	Separate glossary repository	20
4.8	Dependency model	20
5	Packages: datasets and releases	22
5.1	Row meaning	22
5.2	Field reference	22
5.3	Dataset-level vs. release-level metadata	26
5.4	Creating a new dataset concept	27
5.5	Creating a new release	27
5.6	Provenance fields	27
5.7	Temporal coverage vs. acquisition time	28
5.8	Geographic coverage	28
5.9	Rights	28
5.10	Methods and processing	29
5.11	Lineage	29
5.12	Integrity fields	29
5.13	Release status	29
5.14	Notes are important	30
6	Parties: people and organizations	31
6.1	Why parties are separate	31
6.2	Target field reference	31
6.3	Person records	33
6.4	Organization records	33
6.5	<code>party_id</code> and <code>party_iid</code>	33
6.6	Duplicate prevention	34
6.7	Privacy and appropriateness	34
7	Party roles and contributor attribution	35
7.1	Fundamental row rule	35
7.2	Target v1 schema	35
7.3	Dataset-level and release-level contributions	37
7.4	Why IDs rather than readable slugs?	37
7.5	Role schemas	37
7.6	CRedit integration	38
7.7	Contribution degree	39
7.8	Role evidence	39
7.9	Example	39
7.10	What still needs to be implemented for CRedit	40

8	The registry dictionary	41
8.1	The dictionary should define everything in <code>registry/</code>	41
8.2	Dictionary field reference	42
8.3	Self-description	43
8.4	<code>value_domain_type</code>	43
8.4.1	<code>free_text</code>	43
8.4.2	<code>enum</code>	44
8.4.3	<code>vocab</code>	44
8.4.4	<code>pattern</code>	44
8.4.5	<code>range</code>	44
8.5	Inline lists	45
8.6	Null semantics	45
8.6.1	<code>blank</code>	45
8.6.2	<code>nap</code>	45
8.6.3	<code>unk</code>	45
8.6.4	<code>none</code>	46
8.7	Changing the dictionary	46
8.8	<code>registry--vocab-schema.csv</code>	46
9	Controlled vocabularies	47
9.1	Vocabulary architecture	47
9.2	Vocabulary field reference	47
9.3	Why a common schema?	49
9.4	<code>concept_iid</code> vs. <code>term_code</code>	50
9.4.1	<code>term_code</code>	50
9.4.2	<code>concept_iid</code>	50
9.5	Preferred and alternate labels	50
9.6	Definitions and sources	51
9.7	Hierarchy	51
9.8	Lifecycle	51
9.9	Mapping to external vocabularies	52
9.10	Current core vocabularies	52
9.11	<code>registry-table-iid.csv</code>	52
9.12	Creating a new vocabulary	53
10	Identifiers, releases, paths, and manifests	54
10.1	Two kinds of identifiers	54
10.1.1	Opaque IDs	54
10.1.2	Human-readable IIDs	54
10.2	Why both?	55
10.3	Release slugs	55
10.4	Release sequence	55
10.5	UUIDv7 release IDs	55

10.6	Project-relative paths	56
10.7	Raw release layout	56
10.8	SHA-256 manifest	56
10.9	What the integrity fields mean	57
10.9.1	file_manifest_path	57
10.9.2	manifest_sha256	57
10.9.3	file_count	57
10.9.4	total_bytes	57
10.10	What a manifest does not prove	57
10.11	Verify before transforming	58
11	Registering a dataset	59
11.1	Before you begin	59
11.2	Step 1: decide whether this is a new dataset or a new release	59
11.3	Step 2: mint identifiers	60
11.4	Step 3: create the release directory	60
11.5	Step 4: generate the manifest	60
11.6	Step 5: draft the packages row	61
11.7	Step 6: register parties	61
11.8	Step 7: assign roles	61
11.9	Step 8: run validation	62
11.10	Step 9: review against the source	62
11.11	Step 10: commit the registration	62
11.12	Registration is not harmonization	62
12	LLM-assisted first-pass registration	63
12.1	Core rule	63
12.2	What an LLM should read first	63
12.3	Upload/chat prompt	63
12.4	In-repository coding-agent prompt	65
12.5	Review prompt	66
12.6	Evidence discipline	66
12.7	What LLMs are good at here	67
12.8	What requires caution	67
12.9	Final rule	68
13	Validation and review	69
13.1	Machine validation	69
13.1.1	Structure	69
13.1.2	Requiredness	69
13.1.3	Patterns and types	69
13.1.4	Controlled values	70
13.1.5	Referential integrity	70

13.1.6	File integrity	70
13.2	Conditional role validation	70
13.3	Human review	71
13.3.1	Identity review	71
13.3.2	Provenance review	71
13.3.3	Rights review	71
13.3.4	Contributor review	71
13.3.5	Scientific review	71
13.4	Suggested review checklist	72
13.5	Errors vs. warnings	72
13.5.1	Errors	72
13.5.2	Warnings	72
14	Governance and versioning	74
14.1	Separate data changes from schema changes	74
14.2	Proposed change workflow	74
14.3	Preserve backward interpretability	74
14.4	Versioning the registry standard	75
14.5	Git workflow	75
14.6	Vocabulary governance	75
14.6.1	New concept	76
14.6.2	Label refinement	76
14.6.3	Meaning change	76
14.6.4	External standard update	76
14.7	Roles and attribution	76
14.8	Auditability	76
15	Scripts and automation	77
15.1	Core generic tools	77
15.1.1	<code>mint_ids.py</code>	77
15.1.2	<code>manifest.py</code>	77
15.1.3	<code>validate_registry.py</code>	78
15.2	Optional/source-specific tools	78
15.2.1	<code>inventory.py</code>	78
15.2.2	<code>extract_mdb.py</code>	78
15.2.3	<code>extract_props.py</code>	78
15.2.4	<code>convert_map.py</code>	78
15.3	Core vs. optional philosophy	79
15.4	Running scripts from anywhere	79
15.5	Reproducibility	79
16	Worked example	81
16.1	Scenario	81

16.2	Step 1: dataset or release?	81
16.3	Step 2: mint the release	82
16.4	Step 3: preserve raw files	82
16.5	Step 4: generate manifest	82
16.6	Step 5: package metadata	83
16.7	Step 6: parties	83
16.8	Step 7: roles	84
16.9	Step 8: role evidence	84
16.10	Step 9: review uncertainties	85
16.11	Step 10: validate and commit	85
17	Template readiness and remaining implementation work	86
17.1	Architecture decisions already made	86
17.2	Reported/created changes	86
17.3	Registry/dictionary updates still needed	87
17.4	Identifier naming cleanup still needed	87
17.5	Parties updates still needed	88
17.6	Party-role redesign still needed	88
17.7	CRedit integration still needed	88
17.8	File naming cleanup	89
17.9	Script updates	89
17.10	Documentation synchronization	90
17.11	Definition of “ready for lab use”	90

1 Lab Data Registry

The **Lab Data Registry** is a standardized, machine-readable system for documenting datasets that enter research projects in the lab. Its purpose is to make the identity, provenance, content, rights, integrity, and human contributions associated with a dataset explicit **before** the data are harmonized, analyzed, or transformed.

The registry is designed for projects that combine heterogeneous data from multiple sources. Each project can maintain its own populated registry while using the same shared structure, conventions, vocabularies, identifiers, and validation rules.

! Important

The registry CSV files are the **system of record**. This documentation explains the system, but rendered documentation is not authoritative metadata. When the documentation and the registry disagree, the registry specification and versioned schema must be reconciled explicitly.

1.1 What the registry contains

The core registry consists of five standardized CSV artifacts in **registry/**:

1. **registry--packages.csv** — one row per acquired dataset release or snapshot.
2. **registry--parties.csv** — one row per person or organization that must be referenced.
3. **registry--party-roles.csv** — one row per contributor-role assertion.
4. **registry--dictionary.csv** — the field-level specification for every standardized registry CSV.
5. **registry--vocab-schema.csv** — an empty header-only template that defines the physical structure required for controlled-vocabulary CSVs.

Controlled vocabularies live in **vocab/**. The lab glossary is maintained separately in its own repository.

1.2 Start here

If you are new to the registry:

- Read [Overview](#).
- Read [Design principles](#).
- Review the [repository architecture](#).
- Follow [Registering a dataset](#).
- If you are using an LLM for the first pass, use the prompts in [LLM-assisted registration](#).
- Before merging changes, complete [Validation and review](#).

1.3 Core idea in one sentence

Register the dataset you actually received, preserve its raw bytes, document what it is and where it came from, identify the people and organizations involved, record what they contributed, and make every controlled term and field definition explicit.

1.4 Documentation scope

This book documents the **registry standard and workflow**. It does not define project-specific scientific schemas such as soil pedon/horizon tables, vegetation observations, laboratory tables, model outputs, or harmonized analysis products. Projects may document those with the same principles, but they remain project-specific.

The shared **data-management glossary** is intentionally outside this repository so it can evolve as a lab-wide resource used by multiple registry implementations.

2 Overview

2.1 Why a dataset registry?

Research projects often accumulate data in ways that are understandable at the moment of acquisition but difficult to reconstruct later. Files arrive by email, download, shared drive, collaborator handoff, API export, database extract, or legacy archive. Names change. A source may be downloaded more than once. A later release may partly overlap an earlier release. The people who collected or curated the data can disappear from the visible provenance chain once a consolidated dataset is created.

The Lab Data Registry is intended to prevent those problems by requiring a small amount of structured documentation at the point where data enter a project.

The registry answers four foundational questions:

1. **What did we receive?**
2. **Which exact release or snapshot did we use?**
3. **Where did it come from and what are the conditions on its use?**
4. **Who contributed to it, and in what role?**

A fifth component—the registry dictionary—makes the registry itself interpretable by defining the meaning and legal values of every standardized field.

2.2 What is being registered?

The primary object is an **acquired dataset release**.

A dataset is a conceptual data resource that can persist through time. A release is a specific delivered, downloaded, exported, or frozen snapshot of that dataset.

For example, a statewide monitoring dataset downloaded in 2024 and downloaded again in 2026 may be the same dataset concept but two different releases. They should share a stable `dataset_id` and `ds_iid` while receiving distinct `release_id` and `release_slug` values.

This distinction is central to the registry.

2.3 What the registry is not

The registry is not:

- a replacement for the raw data;
- a scientific data model for every project;
- a general-purpose database management system;
- a prose README that happens to list datasets;
- a publication bibliography;
- a substitute for repository metadata such as EML or DataCite;
- a guarantee that all metadata are correct merely because a row exists.

The registry is a structured local record that connects those resources and makes the acquisition and curation process auditable.

2.4 Registry vs. scientific schema

The registry operates primarily at the **dataset/release level**.

A project-specific scientific schema describes the structure of the actual research data: for example pedons, horizons, soil samples, vegetation observations, laboratory results, rasters, or model predictions.

Those two concerns should not be conflated.

A project may use the registry to record an acquired geodatabase, then separately inventory hundreds of fields and map them into a harmonized soil schema. The registry records the acquired package and its provenance; the project schema records how scientific observations are represented.

2.5 Registry vs. glossary

The registry dictionary defines fields such as `release_status`, `dataset_id`, or `source_citation`.

The separate lab glossary defines broader concepts such as *dataset*, *release*, *metadata*, *registry*, *schema*, and *controlled vocabulary*. The glossary is a human knowledge resource shared across projects; the registry is a machine-readable metadata system instantiated separately within each project.

2.6 Intended users

The standard is designed for:

- graduate and undergraduate researchers;
- principal investigators;
- data curators;
- collaborators preparing datasets for integration;
- future lab members who were not present when the data were acquired;
- computational tools and LLMs that need an explicit, inspectable schema.

The system should be usable by a person editing CSV files directly, but structured enough to support validation and automation.

3 Design principles

The registry is built around a small set of rules. These rules are more important than any individual filename.

3.1 Machine-readable first

Registry CSVs and controlled vocabularies are authoritative machine-readable records. Human-readable documentation may be generated from them, but the rendered documentation is a presentation layer.

This means that a fact is not fully registered merely because it appears in a README or manuscript. Important provenance, identity, rights, status, and attribution information should be represented in the structured registry.

3.2 Dataset identity is distinct from release identity

A dataset concept persists across multiple acquisitions or versions. A release represents one specific snapshot.

Stable dataset identity allows the registry to answer:

Are these two deliveries versions of the same conceptual dataset?

Release identity allows the registry to answer:

Are these the exact same bytes or the same specific source snapshot?

Both are necessary.

3.3 Raw acquisitions are immutable

Files placed under a release **raw/** directory should not be edited in place.

If an Access database is exported to CSV, a spreadsheet is cleaned, a geodatabase is normalized, or a text encoding is repaired, that transformation should produce a derived artifact rather than silently altering the source acquisition.

The registry therefore separates **what arrived** from **what we subsequently did to it**.

3.4 Integrity is explicit

Each acquired release should receive a deterministic SHA-256 manifest. The manifest provides a cryptographic fingerprint of the files included in the release.

The registry records:

- the manifest path;
- the SHA-256 of the manifest;
- the file count;
- total bytes.

This allows future users to verify whether a copy of the raw acquisition is exactly the version that was registered.

3.5 Identifiers are stable

Opaque UUIDs and human-readable IIDs serve different purposes.

- *_id fields are stable opaque identifiers used for relationships.
- *_iid fields are human-readable immutable identifiers used for navigation and interpretation.
- release slugs are readable identifiers for specific acquisitions.

An identifier is never recycled for a different concept.

3.6 People and organizations remain visible

Data integration can unintentionally erase the contributions of field crews, laboratory analysts, database maintainers, curators, and source organizations.

The registry therefore separates:

- **party identity** — who the person or organization is;
- **party role** — what that party did in relation to a dataset or release.

This supports richer attribution than placing a single “owner” or “contact” field on a dataset.

3.7 Controlled values are explicit

If a field accepts a defined set of values, those values should not depend on memory or ad hoc spelling.

Small, stable sets may be stored as inline enums. Larger, reusable, hierarchical, externally standardized, or definition-bearing value sets should be represented as vocabulary CSVs.

3.8 Uncertainty is documented, not hidden

A first-pass registry may contain unknowns. That is acceptable.

The correct response to missing evidence is to record the uncertainty using the registry’s null semantics and notes—not to invent a plausible value.

This principle is especially important when LLMs are used to accelerate initial registration.

3.9 Automation assists curation; it does not replace review

LLMs and scripts are encouraged for:

- extracting likely metadata from source documentation;
- profiling files;
- drafting package rows;
- suggesting contributors and roles;
- checking formats;
- generating manifests;
- validating controlled values.

However, the first-pass output must be reviewed by a person who can evaluate source evidence and project context.

3.10 The system should fail visibly

Silent inconsistency is more dangerous than an explicit error.

Validation should detect malformed identifiers, invalid controlled values, orphaned foreign keys, bad paths, missing required fields, and schema drift. When a fact cannot be validated automatically, the documentation should say that human review is required.

4 Repository architecture

4.1 Recommended project tree

The lab-wide template uses descriptive directories without numeric prefixes:

```
project/
  README.md
  project.toml          # optional project marker/configuration
  registry/
    registry--packages.csv
    registry--parties.csv
    registry--party-roles.csv
    registry--dictionary.csv
    registry--vocab-schema.csv
  vocab/
    registry-table-iid.csv
    content-domains.csv
    release-status.csv
    role-schema.csv
    credit-roles.csv
    credit-contribution-degree.csv
    ...
  data/
    <ds_iid>/
      <release_slug>/
        raw/
        extracted/
        reference/
        <release_slug>.sha256
  scripts/
    mint_ids.py
    manifest.py
    validate_registry.py
    ...
  docs/
```

```
index.md
01-overview.md
...
```

The lab glossary lives in a **separate repository** and is not part of this project tree.

4.2 registry/

The **registry/** directory contains the standardized tables that define and populate the project registry.

A useful rule is:

Every standardized CSV in **registry/** must be defined by **registry--dictionary.csv**.

The five canonical registry artifacts are:

4.2.1 registry--packages.csv

Dataset- and release-level descriptive metadata, provenance, coverage, rights, processing, integrity, and status.

4.2.2 registry--parties.csv

Stable identities for people and organizations referenced by the registry.

4.2.3 registry--party-roles.csv

Relationships between parties and datasets/releases. One row should represent one role assertion.

4.2.4 registry--dictionary.csv

Field-level specification for all standardized registry tables, including itself and the vocabulary schema.

4.2.5 registry--vocab-schema.csv

An intentionally empty, header-only CSV that physically represents the common schema all vocabulary files must follow.

It is not a vocabulary. It is a standardized registry artifact whose columns are defined in the dictionary.

4.3 vocab/

The `vocab/` directory contains controlled value sets used by registry fields.

All files in this directory should conform to the header in `registry--vocab-schema.csv`.

Examples include:

- `registry-table-iid.csv`;
- `content-domains.csv`;
- `release-status.csv`;
- `role-schema.csv`;
- `credit-roles.csv`;
- `credit-contribution-degree.csv`.

A vocabulary's filename is descriptive; the standardized structure comes from the vocab schema.

4.4 data/

The `data/` directory contains acquired releases and derived material.

```
data/<ds_iid>/<release_slug>/
```

creates a direct mapping between registry identities and filesystem locations.

A release directory may contain:

- `raw/` — unchanged acquired files;
- `extracted/` — machine-generated exports from source containers or databases;
- `reference/` — documentation supplied with the release or curated reference material;
- curation tables or inventories;
- a manifest beside the directories it describes.

Projects may add additional derived directories, but raw acquisitions should remain immutable.

4.5 scripts/

Core scripts implement repeatable registry operations.

The shared standard should keep only genuinely generic tools here. Highly source-specific tooling can live in project repositories or an optional utilities area.

Core examples:

- mint dataset, release, and party identifiers;
- create and verify manifests;
- validate registry CSVs against the dictionary;
- validate foreign keys and vocabularies.

4.6 docs/

The docs/ directory contains this Quarto-ready handbook.

The documentation explains the standard, but does not replace the machine-readable registry.

4.7 Separate glossary repository

The shared lab glossary is maintained independently because it has a different lifecycle from project registries.

Projects can link to the glossary repository in documentation without copying all glossary content into each registry. This avoids divergent definitions and allows terms to evolve through one shared Git history.

4.8 Dependency model

The architecture can be summarized as:

```
registry--dictionary.csv
    defines registry--packages.csv
    defines registry--parties.csv
    defines registry--party-roles.csv
    defines itself
```

```
defines registry--vocab-schema.csv
```

```
constrains every CSV in vocab/
```

Project-specific scientific data structures sit outside this core and can build on it without changing the shared registry standard.

5 Packages: datasets and releases

`registry--packages.csv` is the central dataset-level inventory.

5.1 Row meaning

One row represents **one acquired release or snapshot of one dataset concept**.

This means that repeated releases of the same dataset produce repeated rows with the same:

- `dataset_id`;
- `ds_iid`;

but different:

- `release_id`;
- `release_slug`;
- release-specific dates, source details, processing notes, and integrity fields.

5.2 Field reference

Field	Required	Type/domain	Meaning
<code>dataset_id</code>	yes	string / UUIDv4	Stable opaque identifier for the dataset concept. Copied unchanged to every release row for that dataset.
<code>ds_iid</code>	yes	string / IID	Stable human-readable dataset identifier in lowercase kebab-case. Immutable and never reused.

Field	Required	Type/domain	Meaning
release_id	conditional	string / UUIDv7	Opaque identifier for a specific acquired release or snapshot. Required for a registered release.
release_slug	conditional	string / pattern	Human-readable release identifier: {ds_iid}--ingYYYYMMDD--rNNN.
title	yes	free text	Human-readable title for this dataset release. Prefer source-provided wording when available.
abstract	yes	free text	Concise description of the content and scope of the release.
purpose	no	free text	Why the source dataset was created or why it is being incorporated into the project.
source_org	yes	free text	Organization primarily responsible for the source delivery or source system.
source_citation	yes	free text	Citation or provenance statement describing the source of this release.
landing_page_url	no	URL/text	Public or internal landing page for the source dataset when one exists.
doi	no	identifier/text	DOI associated with the source dataset or release, if applicable.

Field	Required	Type/domain	Meaning
eml_package_id	no	identifier/text	EML package identifier when the source uses Ecological Metadata Language.
repo_scope	no	free text / controlled later	Repository or archive system in which the source object is registered.
repo_identifier	no	free text	Identifier assigned by the source repository or archive.
repo_revision	no	free text	Revision/version identifier used by the source repository.
coverage_start	conditional	date/datetime	Beginning of the temporal coverage represented by the data, when applicable.
coverage_end	conditional	date/datetime	End of temporal coverage represented by the data, when applicable.
published_date	no	date	Date the source release was published, when known.
ingest_date	conditional	date	Date this release entered the local registry/workflow. Drives the readable release slug.
geo_description	conditional	free text	Human-readable spatial coverage description.
west	no	float	Western longitude/bounding coordinate when a bounding box is useful.

Field	Required	Type/domain	Meaning
east	no	float	Eastern longitude/bounding coordinate.
south	no	float	Southern latitude/bounding coordinate.
north	no	float	Northern latitude/bounding coordinate.
spatial_reference	no	free text	Spatial reference system or coordinate reference description when applicable.
content_domains	conditional	vocab / multivalue	High-level content categories governed by <code>vocab/content-domains.csv</code> . Multiple values use an unpadding pipe.
license	conditional	free text	Formal license governing reuse, if stated.
intellectual_rights	conditional	free text	Rights, restrictions, data-sharing conditions, or intellectual-property notes not captured by <code>license</code> .
methods_summary	conditional	free text	Brief summary of how the source data were collected or generated.
processing_summary	conditional	free text	Brief summary of transformations already applied before or during ingest.

Field	Required	Type/domain	Meaning
upstream_release_ids	no	multivalued IDs	Release IDs that are direct ancestors or upstream inputs. Do not use merely to indicate thematic similarity.
file_manifest_path	conditional	project-relative path	Path to the release manifest. Use a project-relative POSIX path.
manifest_sha256	conditional	SHA-256 hex	SHA-256 digest of the manifest file itself.
file_count	conditional	integer	Number of files represented by the manifest.
total_bytes	conditional	integer	Total byte size of all files represented by the manifest.
release_status	yes	vocab	Lifecycle/registration status governed by <code>vocab/release-status.csv</code> .
notes	no	free text	Curator notes, caveats, unresolved issues, and contextual information that do not fit another field.

5.3 Dataset-level vs. release-level metadata

Some package fields describe the stable conceptual dataset; others may vary by release.

In practice, the packages table intentionally stores both in one release-grained table because a self-contained release row is easier to archive, export, validate, and interpret than a heavily normalized metadata database.

When a value is stable across releases, copy it consistently. When a later release changes scope, rights, source system, processing, or coverage, record the release-specific truth rather than forcing consistency.

5.4 Creating a new dataset concept

Create a new `dataset_id` and `ds_iid` when the source represents a genuinely distinct conceptual dataset.

Do **not** create a new dataset merely because:

- you downloaded it again;
- a new annual release appeared;
- the file format changed;
- the source repository issued a new revision;
- additional files were added to a later snapshot.

Those situations generally indicate a new release of the existing dataset.

5.5 Creating a new release

A new release is appropriate when the actual acquired content has changed or a new source snapshot must be distinguished from the prior one.

Examples:

- annual or periodic source updates;
- a collaborator sends a corrected delivery;
- the source repository republishes a new version;
- you intentionally preserve a later database export;
- the same named resource is acquired at a different time and differs in content.

5.6 Provenance fields

`source_org`, `source_citation`, `repository` fields, URLs, and identifiers should reflect the source evidence.

Prefer:

1. a source-provided citation;
2. repository metadata;
3. accompanying documentation;
4. a curator-written provenance statement when no formal citation exists.

Do not make a citation look more formal than the evidence supports.

5.7 Temporal coverage vs. acquisition time

Do not confuse:

- `coverage_start` / `coverage_end` — when the observations or represented phenomena occurred;
- `published_date` — when the source release was published;
- `ingest_date` — when this release entered the local workflow.

These dates can be very different.

5.8 Geographic coverage

Use `geo_description` for a human-readable description.

The bounding fields are optional. They are most useful when a simple rectangular extent is meaningful and the coordinates can be supported by the source or a reproducible calculation.

Do not invent a bounding box from a vague place name merely to fill four fields.

5.9 Rights

`license` records a formal license when one exists.

`intellectual_rights` captures broader restrictions or conditions such as:

- redistribution limitations;
- attribution requirements;
- agreements with collaborators;
- source-agency conditions;
- uncertainty about rights.

If rights are unclear, document that uncertainty.

5.10 Methods and processing

`methods_summary` describes how the source data were produced.

`processing_summary` describes what happened to the data before or during acquisition/ingest.

This distinction is useful because a dataset may have been collected with one method, exported through another system, cleaned by a source collaborator, and then locally extracted without altering the original release.

5.11 Lineage

Use `upstream_release_ids` only for actual lineage relationships.

A prior release is not automatically an upstream ancestor. For example, two independent exports from the same external database may overlap without one being derived from the other.

Document ambiguous lineage in `notes`.

5.12 Integrity fields

The four integrity fields are normally script-generated:

- `file_manifest_path`;
- `manifest_sha256`;
- `file_count`;
- `total_bytes`.

Do not hand-calculate them when the manifest script is available.

5.13 Release status

`release_status` is controlled by `vocab/release-status.csv`.

The status vocabulary should express the local registration/curation workflow. A status such as `registered` should have a precise definition—for example, required metadata completed, raw release preserved, manifest generated, and required relationships validated.

5.14 Notes are important

The `notes` field is not a dumping ground, but it is intentionally flexible.

Use it for information that would otherwise be lost, such as:

- partial supersession;
- scope caveats;
- promises of follow-on deliveries that never arrived;
- conflicting dates;
- naming mismatches between source systems;
- unresolved rights questions;
- known limitations that should be revisited during harmonization.

6 Parties: people and organizations

`registry--parties.csv` gives stable identities to the people and organizations referenced elsewhere in the registry.

6.1 Why parties are separate

A person's identity is not the same thing as their contribution.

For example, a person may be:

- a field investigator for one dataset;
- a curator for another;
- an author or data manager for a third;
- a supervisor at the dataset level but not involved in a particular release.

The parties table therefore answers:

Who is this?

The party-roles table answers:

What did this party do, for which dataset or release?

6.2 Target field reference

Field	Required	Type/domain	Meaning
party_id	yes	string / UUIDv4	Stable opaque identifier for a person or organization.

Field	Required	Type/domain	Meaning
party_iid	yes	string / IID	Stable human-readable identifier in lowercase kebab-case; immutable and never reused.
type	yes	enum	Whether the party is a person or organization .
given_name	conditional	free text	Given name for a person.
middle_name	no	free text	Middle name or initials for a person.
surname	conditional	free text	Surname/family name for a person.
org_name	conditional	free text	Official or preferred organization name when the party is an organization.
email	no	free text	Contact email when appropriate to record and share.
orcid	no	ORCID pattern	ORCID identifier for a person when available.
organization	no	free text	Primary affiliation or organization for a person, when useful.
notes	no	free text	Identity clarification, alternate forms, affiliation caveats, or other curation notes.

i Note

The intended v1 schema removes `role_default`. Roles should not be treated as intrinsic properties of a party; context-specific contribution belongs in `registry--party-roles.csv`.

6.3 Person records

For a person:

- `type=person`;
- use the person's preferred or documented name form;
- record ORCID when known and appropriate;
- use **organization** for affiliation if useful;
- use notes to document ambiguous identity, name variants, or uncertain affiliation.

Do not fabricate middle names, email addresses, ORCIDs, or institutional affiliations.

6.4 Organization records

For an organization:

- `type=organization`;
- populate `org_name`;
- leave person-name fields not applicable according to the registry null convention;
- create a stable `party_id` and `party_iid` just as for a person.

Organizations may themselves have roles in relation to datasets.

6.5 `party_id` and `party_iid`

`party_id` is the stable opaque identifier used for relationships.

`party_iid` is the readable immutable identifier.

Good IIDs are:

- lowercase;
- short but recognizable;
- kebab-case;
- stable even if a person's affiliation changes.

Examples:

```
njelinski  
clping  
alaska-center-conservation-science  
usda-nrcs
```

An IID is not intended to store a person’s current job title or organization.

6.6 Duplicate prevention

Before creating a party, search existing party records.

The same person should not receive separate identities merely because:

- their name is rendered differently in different publications;
- one source uses initials and another uses the full name;
- their affiliation changes;
- one source includes an ORCID and another does not.

If two records may represent the same person but the evidence is uncertain, keep the uncertainty visible rather than merging aggressively.

6.7 Privacy and appropriateness

The registry should contain only contact/identity information that is appropriate for the research context.

An email address is optional. Public persistent identifiers such as ORCID are generally preferable for long-term identity resolution when available.

7 Party roles and contributor attribution

`registry--party-roles.csv` records the relationship between a party and a dataset or release.

7.1 Fundamental row rule

One row = one party × one dataset/release scope × one role assertion.

Do not store several roles in a pipe-delimited `role_code` cell.

If one contributor performed three CRediT roles, create three rows.

This makes role assertions independently searchable, attributable, reviewable, and citable.

7.2 Target v1 schema

Field	Required	Type/domain	Meaning
<code>role_assignment_id</code>	yes	string / UUIDv4	Stable opaque identifier for this one role assertion.
<code>dataset_id</code>	yes	foreign key	Dataset concept to which the contribution applies.
<code>release_id</code>	no	foreign key	Specific release to which the contribution applies. Blank means the assertion applies at the dataset level.
<code>party_id</code>	yes	foreign key	Person or organization performing the role.

Field	Required	Type/domain	Meaning
role_schema	yes	vocab	Role system being used, e.g. credit , datacite , eml , or a designated local schema.
role_code	yes	conditional vocab	Machine-readable role code valid under the selected role_schema . One row contains one role.
contribution_degree	no	conditional vocab	Optional degree such as CRediT lead , equal , or supporting when supported by the role schema.
role_basis	no	vocab / proposed	How the role was established, e.g. declared by source, publication, curator-assigned, or inferred.
role_source	no	free text / path / identifier	Citation, URL, file path, or note pointing to the evidence for the role assertion.
notes	no	free text	Additional context about the role assertion.

Warning

The current legacy party-role CSV and dictionary have not yet been migrated to this target schema. This chapter documents the recommended v1 design. Complete the migration before tagging the registry template as v1.0.

7.3 Dataset-level and release-level contributions

`dataset_id` is always required.

`release_id` is optional:

- blank `release_id` means the role applies to the dataset concept generally;
- populated `release_id` means the role applies specifically to that release.

This avoids a separate `role_scope` field while preserving an important distinction.

7.4 Why IDs rather than readable slugs?

Foreign-key relationships should use stable opaque identifiers:

- `dataset_id`;
- `release_id`;
- `party_id`.

Readable IIDs and slugs are still available in the related tables and can be joined for display.

This avoids duplicating human-readable identifiers throughout relationship tables and reduces the chance that a spelling change breaks referential integrity.

7.5 Role schemas

`role_schema` identifies the controlled role system being used.

Examples may include:

- `credit`;
- `datacite`;
- `eml`;
- a deliberately defined local role schema.

The legal `role_schema` values live in `vocab/role-schema.csv`.

`role_code` must then be validated against the vocabulary associated with that schema.

This is a **conditional vocabulary relationship**.

For example:

```
role_schema = credit
role_code    = data-curation
```

means that `data-curation` must exist in `vocab/credit-roles.csv`.

7.6 CRediT integration

CRediT is the Contributor Role Taxonomy standardized as ANSI/NISO Z39.104-2022. It defines 14 contributor roles and allows a contributor to hold multiple roles. When multiple contributors share a role, an optional degree of contribution may be specified as `lead`, `equal`, or `supporting`.

The registry should include:

```
vocab/credit-roles.csv
vocab/credit-contribution-degree.csv
```

The canonical CRediT role vocabulary contains:

- Conceptualization
- Data curation
- Formal analysis
- Funding acquisition
- Investigation
- Methodology
- Project administration
- Resources
- Software
- Supervision
- Validation
- Visualization
- Writing – original draft
- Writing – review & editing

For external standards, preserve canonical preferred labels and definitions in the vocabulary while using a local machine-readable `term_code`.

Canonical references:

- <https://credit.niso.org/>
- <https://credit.niso.org/contributor-roles-defined/>
- <https://credit.niso.org/contributor-roles/>

7.7 Contribution degree

For CRediT, `contribution_degree` may be:

```
lead  
equal  
supporting
```

The field is optional. It should not be populated for a role schema that does not support the concept unless the local standard explicitly defines how to interpret it.

7.8 Role evidence

A structured contributor system is much stronger if the source of a role assertion is visible.

The target design therefore includes:

- `role_basis` — how the role was established;
- `role_source` — evidence or reference.

A future `role-basis.csv` vocabulary might include values such as:

```
source-declared  
contributor-declared  
publication  
curator-assigned  
inferred
```

The final terms should be reviewed before the vocabulary is adopted.

7.9 Example

A contributor who led field collection and equally contributed to methodology would receive two rows:

```
<uuid-1>,<dataset-id>,,<party-id>,credit,investigation,lead,publication,<doi>,...  
<uuid-2>,<dataset-id>,,<party-id>,credit,methodology,equal,publication,<doi>,...
```

7.10 What still needs to be implemented for CRediT

Before the template is complete:

1. Place `credit-roles.csv` in `vocab/`.
2. Place `credit-contribution-degree.csv` in `vocab/`.
3. Ensure `role-schema.csv` contains the `credit` term and identifies CRediT as the intended scheme.
4. Update the party-role section of `registry--dictionary.csv` to the target schema.
5. Remove `role_default` from the parties schema.
6. Update validation so `role_code` is checked against the vocabulary associated with `role_schema`.
7. Validate `contribution_degree` against the CRediT degree vocabulary when `role_schema=credit`.
8. Decide whether to adopt a `role-basis.csv` vocabulary.

8 The registry dictionary

`registry--dictionary.csv` is the normative field-level specification for the standardized registry.

It answers:

- What columns exist?
- In what order?
- Are they required?
- What physical type is expected?
- How are values constrained?
- Which vocabulary governs a controlled field?
- What does the field mean?

8.1 The dictionary should define everything in `registry/`

The intended standard has dictionary sections for:

```
packages
parties
party-roles
dictionary
vocab-schema
```

The value of `table_iid` is governed by:

```
vocab/registry-table-iid.csv
```

This creates a direct one-to-one mapping between standardized registry artifacts and dictionary table identifiers.

8.2 Dictionary field reference

Field	Required	Type/domain	Meaning
table_iid	yes	vocab	Canonical registry table/artifact identifier. Governed by <code>vocab/registry-table-iid.csv</code> .
column_iid	yes	pattern	Exact machine-readable column name as it appears in the CSV header.
column_label	yes	free text	Human-readable label for the column.
ordinal	yes	integer	One-based column position in the physical CSV.
required	yes	enum	Requirement state such as yes , no , or conditional .
storage_type	yes	enum	Expected physical/storage type, e.g. string, integer, float, datetime.
value_domain_type	yes	enum	How legal values are constrained: free_text , enum , vocab , range , or pattern .
allowed_values	conditional	free text	Inline values, regular expression, or range expression when the constraint is stored directly.
vocab_ref	conditional	path/filename	Vocabulary file used when <code>value_domain_type=vocab</code> .

Field	Required	Type/domain	Meaning
null_tokens	yes	free text	Allowed null semantics for the field. Registry convention distinguishes blank, nap , unk , and dictionary-only none .
description	yes	free text	Normative definition of the field and how it should be interpreted.
notes	no	free text	Additional implementation guidance, examples, or caveats.

8.3 Self-description

The dictionary is self-describing.

Rows where:

```
table_iid = dictionary
```

define the columns of `registry--dictionary.csv` itself.

This is intentionally recursive: the same mechanism used to define **packages** also defines the dictionary.

8.4 value_domain_type

The value-domain type tells validators and curators how to interpret constraints.

8.4.1 free_text

Human-readable prose is permitted.

Use this only when a field genuinely needs unconstrained text.

8.4.2 enum

A small finite list is stored directly in `allowed_values`.

Example:

```
person|organization
```

8.4.3 vocab

The legal values come from a maintained vocabulary file referenced in `vocab_ref`.

Example:

```
vocab_ref = release-status.csv
```

8.4.4 pattern

The value must match a defined syntax, usually represented as a regular expression or documented pattern.

Useful for:

- UUID formatting;
- dates;
- IIDs;
- SHA-256 strings;
- release slugs.

8.4.5 range

The value must fall within a numeric or ordinal range.

8.5 Inline lists

When multiple allowed values are encoded in one dictionary cell, use an **unpadded pipe**:

```
yes|no|conditional
```

not:

```
yes | no | conditional
```

This convention is also used for intentionally multivalued registry cells where the schema allows them.

8.6 Null semantics

The registry distinguishes several states.

8.6.1 blank

Not yet assessed, not yet entered, or temporarily unresolved during drafting.

Blank should generally disappear from fields that are required before registration is complete.

8.6.2 nap

Not applicable.

Example: a DOI field for a locally created dataset that has never had a DOI.

8.6.3 unk

Applicable, but unknown after review.

Example: publication date known to exist but not recoverable from available documentation.

8.6.4 none

Used in the dictionary to mean that a field is not permitted to be null.

Do not use ad hoc null strings such as:

```
NA  
N/A  
NULL  
None  
-9999
```

8.7 Changing the dictionary

Changing the dictionary changes the registry standard.

Therefore:

1. propose the change;
2. explain why it is needed;
3. update the dictionary;
4. update the physical CSV header/template;
5. update relevant vocabularies;
6. update validation;
7. migrate existing rows if necessary;
8. document the schema change;
9. review before merge.

A dictionary change should never be treated as a casual spreadsheet edit.

8.8 registry--vocab-schema.csv

The vocabulary schema is a standardized registry artifact and should therefore be represented in the dictionary under:

```
table_iid = vocab-schema
```

The 16 vocabulary-field definition rows belong inside the main dictionary. A separate `dictionary--vocab-fields.csv` is useful only as a development fragment and should not be the canonical source once those rows have been merged.

9 Controlled vocabularies

Controlled vocabularies make legal values explicit and reusable.

9.1 Vocabulary architecture

Every vocabulary CSV in `vocab/` conforms to:

```
registry/registry--vocab-schema.csv
```

That file contains a header only and no term rows.

Its fields are defined in:

```
registry/registry--dictionary.csv
```

under:

```
table_iid = vocab-schema
```

This creates one standardized vocabulary structure for the entire registry.

9.2 Vocabulary field reference

Field	Required	Type/domain	Meaning
concept_iid	yes	pattern	Stable identifier for the concept represented by the vocabulary row. Recommended form: {vocab_iid}--{term_code}.

Field	Required	Type/domain	Meaning
term_code	yes	pattern	Machine-readable value used in governed registry/data fields. Lowercase kebab-case for local vocabularies.
pref_label	yes	free text	Preferred human-readable label.
alt_labels	no	multivalue text	Alternative labels, abbreviations, synonyms, or source-specific spellings separated by an unpadded pipe.
definition	yes	free text	Definition of the concept. Use canonical wording when adopting an external standard.
broader_id	no	concept ID	Immediate broader/parent concept for hierarchical vocabularies.
status	yes	enum	Concept lifecycle state, typically active , proposed , or deprecated .
created	yes	date	Date the vocabulary row was created, YYYY-MM-DD.
modified	yes	date	Date the vocabulary row was last substantively modified, YYYY-MM-DD.
scope_note	no	free text	Guidance on when or how the concept should be used.

Field	Required	Type/domain	Meaning
related_ids	no	multivalue IDs	Related local concept identifiers that are associative rather than hierarchical/equivalent.
exact_match_ids	no	multivalue IDs/URIs	External identifiers judged to represent the same concept.
close_match_ids	no	multivalue IDs/URIs	External identifiers representing closely related but non-identical concepts.
replaced_by	conditional	concept ID	Preferred successor when a concept is deprecated.
source	yes	free text / citation / URL	Authoritative source for the term or definition.
note	no	free text	Curator comments that do not belong in the definition or scope note.

9.3 Why a common schema?

A vocabulary is more than a list of strings.

The common structure allows the registry to represent:

- stable concept identity;
- machine codes;
- human labels;
- synonyms;
- definitions;
- hierarchy;
- lifecycle state;
- external mappings;
- provenance.

That becomes especially valuable for large scientific classifications and externally maintained standards.

9.4 `concept_iid` vs. `term_code`

These two fields serve different purposes.

9.4.1 `term_code`

The machine-readable value stored in a governed field.

Example:

```
data-curation
```

9.4.2 `concept_iid`

A stable identifier for the vocabulary concept itself.

Recommended local form:

```
credit-role--data-curation
```

A term code may be readable, but the concept ID makes relationships and mappings more explicit.

9.5 Preferred and alternate labels

`pref_label` is the canonical display label.

`alt_labels` can contain synonyms, abbreviations, older labels, or source-specific variants.

Multiple alternative labels use an unpadded pipe:

```
data management|research data curation
```

Alternative labels do not become additional legal term codes unless the vocabulary explicitly creates them as distinct concepts.

9.6 Definitions and sources

For an externally maintained standard such as CRediT:

- preserve the canonical preferred label;
- preserve or accurately reproduce the canonical definition;
- cite the canonical source in **source**;
- keep local implementation details in **scope_note** or **note**.

Do not silently rewrite an external standard merely to fit local terminology.

9.7 Hierarchy

broader_id allows a vocabulary to represent parent-child relationships.

This is useful for hierarchical concepts such as:

- soil taxonomy;
- geomorphic classifications;
- organizational hierarchies;
- method families.

A flat vocabulary simply leaves **broader_id** empty/not applicable.

9.8 Lifecycle

Terms can evolve.

Use **status** to distinguish:

```
proposed
active
deprecated
```

When a deprecated term has a direct successor, populate **replaced_by**.

Do not delete a published vocabulary term merely because it is no longer preferred; historical data may still contain its code.

9.9 Mapping to external vocabularies

`exact_match_ids` and `close_match_ids` allow local concepts to be linked to external identifiers.

Use exact matches conservatively. “Looks similar” is not the same as semantic equivalence.

9.10 Current core vocabularies

The registry currently anticipates at least:

```
registry-table-iid.csv
content-domains.csv
release-status.csv
role-schema.csv
credit-roles.csv
credit-contribution-degree.csv
```

Additional vocabularies should be created only when a controlled field actually requires them.

9.11 registry-table-iid.csv

This vocabulary defines the standardized registry table identifiers.

The intended terms are:

```
packages
parties
party-roles
dictionary
vocab-schema
```

These correspond to the five canonical CSV artifacts in `registry/`.

9.12 Creating a new vocabulary

1. Copy the header from `registry--vocab-schema.csv`.
2. Choose a descriptive kebab-case filename.
3. Define the legal `term_code` values.
4. Create stable `concept_iid` values.
5. Add preferred labels and definitions.
6. Record the source.
7. Add hierarchy/mappings only when supported.
8. Reference the vocabulary from the relevant dictionary row using `vocab_ref`.
9. Update validation if special conditional rules are required.
10. Review the vocabulary before it becomes authoritative.

10 Identifiers, releases, paths, and manifests

This chapter describes the identity and integrity conventions that make the registry reproducible.

10.1 Two kinds of identifiers

10.1.1 Opaque IDs

Opaque `*_id` fields are primarily for machine relationships.

The target conventions are:

```
dataset_id      UUIDv4
party_id        UUIDv4
role_assignment_id UUIDv4
release_id      UUIDv7
```

UUID versions are implementation choices documented by the standard; the UUID type does not belong in the column name.

10.1.2 Human-readable IIDs

Human-readable immutable identifiers use the `_iid` suffix.

Examples:

```
ds_iid      = blm-aim
party_iid   = njelinski
```

IIDs use lowercase kebab-case and should remain stable.

10.2 Why both?

Opaque IDs are robust relationship keys.

Readable IIDs are easier for humans to navigate, type, search, and recognize.

The registry intentionally uses both rather than forcing one identifier to do both jobs.

10.3 Release slugs

A release slug combines:

- dataset IID;
- ingest date;
- release sequence.

Pattern:

```
{ds_iid}--ing{YYYYMMDD}--r{NNN}
```

Example:

```
blm-aim--ing20260914--r003
```

The double dash separates semantic components. Internal word separators use a single hyphen.

10.4 Release sequence

r001, r002, etc. are local release counters.

They are not intended to reproduce a source repository's version numbering. Source versions belong in the relevant repository/version fields.

10.5 UUIDv7 release IDs

UUIDv7 provides a time-sortable identifier.

The current minting approach embeds the delivery/ingest day in the UUIDv7 timestamp. The readable release slug remains the authoritative human sequence, especially when multiple releases are registered on the same day.

10.6 Project-relative paths

Paths recorded in registry metadata should be:

- relative to the project root;
- POSIX-style with /;
- independent of a particular user's workstation.

Good:

```
data/blm-aim/blm-aim--ing20260914--r003/blm-aim--ing20260914--r003.sha256
```

Bad:

```
/Users/nic/Desktop/project/data/...  
C:\Users\someone\project\data\...
```

Absolute local paths make a registry non-portable.

10.7 Raw release layout

Recommended layout:

```
data/  
  blm-aim/  
    blm-aim--ing20260914--r003/  
      raw/  
        extracted/  
        reference/  
        blm-aim--ing20260914--r003.sha256
```

10.8 SHA-256 manifest

The manifest contains one deterministic line per included file:

```
<sha256> <relative/path/inside/raw>
```

The entries are sorted by path.

A cryptographic hash changes when file content changes, making the manifest a strong way to detect whether a supposedly identical release has been altered.

10.9 What the integrity fields mean

10.9.1 `file_manifest_path`

Where the manifest is located, relative to the project root.

10.9.2 `manifest_sha256`

SHA-256 digest of the manifest itself.

If two parties have the same manifest digest, the manifest contents are byte-for-byte identical.

10.9.3 `file_count`

Number of files represented.

10.9.4 `total_bytes`

Combined byte size of the represented files.

10.10 What a manifest does not prove

A manifest proves byte identity, not scientific correctness.

It does not establish that:

- the source was authoritative;
- the files are complete;
- the observations are accurate;
- the license is correct;
- the extraction was scientifically appropriate.

Those questions require provenance and review.

10.11 Verify before transforming

When possible:

1. acquire the release;
2. freeze it under **raw**/;
3. create the manifest;
4. register integrity metadata;
5. only then begin extraction or transformation.

This sequence preserves a defensible baseline.

11 Registering a dataset

This chapter is the standard end-to-end workflow.

11.1 Before you begin

Gather everything that came with the dataset:

- raw files;
- README files;
- metadata exports;
- database dictionaries;
- source URLs;
- citations;
- emails or handoff notes that contain provenance;
- licenses or data-use conditions;
- documentation naming contributors or organizations.

Do not start by filling cells from memory.

11.2 Step 1: decide whether this is a new dataset or a new release

Ask:

Does this acquisition represent a new conceptual data resource, or another snapshot/version of a dataset already registered?

If new, mint a new dataset identity.

If it is another release, reuse the existing `dataset_id` and `ds_iid`.

When uncertain, document the uncertainty before minting duplicate concepts.

11.3 Step 2: mint identifiers

For a new dataset:

```
python scripts/mint_ids.py dataset my-dataset
```

For a release:

```
python scripts/mint_ids.py release my-dataset <existing-dataset-id> --ingest 2026-09-1
```

For a new party:

```
python scripts/mint_ids.py party jane-smith
```

Do not ask an LLM to invent UUIDs in prose when the project script is available.

11.4 Step 3: create the release directory

Create:

```
data/<ds_iid>/<release_slug>/raw/
```

Place an unchanged copy of the acquired release in **raw/**.

Preserve source filenames unless there is a compelling archival reason to wrap the delivery in another container. Renaming raw files can make later source comparison harder.

11.5 Step 4: generate the manifest

Run the project manifest tool.

For example:

```
python scripts/manifest.py data/<ds_iid>/<release_slug>/raw
```

Record the generated integrity fields in the package row.

11.6 Step 5: draft the packages row

Use source evidence to fill:

- description;
- provenance;
- source identifiers;
- dates;
- geographic coverage;
- rights;
- methods;
- processing;
- lineage;
- integrity;
- status.

Use the dictionary while drafting. Do not assume a field's meaning from its name alone.

11.7 Step 6: register parties

Identify people and organizations that need stable references.

Before creating a new party, search `registry--parties.csv` for an existing record.

Add only identity information supported by evidence.

11.8 Step 7: assign roles

For each contributor relationship:

1. determine dataset-level or release-level scope;
2. choose the role schema;
3. choose one valid role code;
4. create one role row;
5. optionally record degree and evidence.

Do not compress several roles into one cell.

11.9 Step 8: run validation

Run:

```
python scripts/validate_registry.py
```

Fix machine-detectable errors before requesting human review.

11.10 Step 9: review against the source

A second pass should verify:

- identity;
- release/dataset distinction;
- dates;
- provenance;
- rights;
- citations;
- contributors;
- uncertain values;
- manifest information.

11.11 Step 10: commit the registration

Commit the registry changes, vocab changes if any, manifests, and any new scripts or documentation together when they represent one logical registration event.

A useful commit message is specific:

```
Register BLM AIM release ing20260915 r003
```

11.12 Registration is not harmonization

Do not delay registration until scientific cleaning is complete.

The registry should capture the acquisition **before** the messy work of schema harmonization begins. Later processing can be documented incrementally while preserving the original source context.

12 LLM-assisted first-pass registration

LLMs can dramatically reduce the time required to assemble a first-pass registry entry, especially when a release includes multiple documentation files. They should be used as **curation assistants**, not as authorities.

12.1 Core rule

Extract, infer cautiously, and flag uncertainty. Never fabricate missing metadata to make the row look complete.

An LLM-generated registry row is a draft until reviewed by a person.

12.2 What an LLM should read first

Before analyzing a dataset, provide or direct the agent to:

1. `registry/registry--dictionary.csv`;
2. `registry/registry--packages.csv`;
3. `registry/registry--parties.csv`;
4. `registry/registry--party-roles.csv`;
5. `registry/registry--vocab-schema.csv`;
6. all relevant files in `vocab/`;
7. this documentation;
8. source documentation supplied with the dataset.

This prevents the model from inventing a parallel schema.

12.3 Upload/chat prompt

Use this when working in ChatGPT, Claude, Gemini, or another chat interface where you are uploading the dataset documentation.

I am registering a dataset using our lab's standardized data registry.

Attached are:

1. the source dataset files and/or documentation available to me;
2. registry--dictionary.csv;
3. the current packages, parties, and party-roles CSVs;
4. the relevant vocabularies.

Your job is to prepare a FIRST-PASS registration for human review.

Rules:

- Treat registry--dictionary.csv as authoritative for field meaning and constraints.
- Do not add, remove, rename, or reorder registry columns.
- Do not invent facts that are not supported by the source material.
- Distinguish explicitly among:
 - (a) values directly supported by source evidence,
 - (b) reasonable inferences,
 - (c) unknown or unresolved values.
- Use our null semantics correctly: blank for not-yet-assessed during drafting, nap for not applicable, and unk for applicable but unknown after review.
- Use only legal vocabulary term_code values from the supplied vocab files.
- For intentional multivalued cells, use an unpadded pipe: a|b|c.
- Do not create a new dataset concept until you compare the acquisition with existing packages rows and determine whether it is a new dataset or a new release.
- Do not invent UUIDs. Tell me which identifiers need to be minted with the project script.
- Do not modify raw source files.
- Preserve source-provided citations, names, identifiers, dates, and rights language as faithfully as possible.
- Flag conflicting evidence rather than silently choosing one source.
- Search the existing parties table before proposing a new party.
- For contributor roles, propose one row per role assertion and use an existing role schema/

Return:

- A. A short assessment: new dataset concept or new release, with reasoning.
- B. A proposed packages row in exact CSV column order.
- C. Proposed new parties, if any.
- D. Proposed party-role assertions, if supported.
- E. A list of unresolved questions / fields needing human review.
- F. A source-evidence table showing where the most important values came from.
- G. A list of any vocabulary terms or schema changes you think may be needed. Do not make those

12.4 In-repository coding-agent prompt

Use this with Codex, Claude Code, or another agent operating inside the project directory.

You are helping prepare a FIRST-PASS dataset registration in this repository.

Before making any changes:

1. Read the documentation under docs/.
2. Read registry/registry--dictionary.csv.
3. Read all existing registry CSVs.
4. Read registry/registry--vocab-schema.csv and all vocabularies in vocab/.
5. Inspect the source dataset delivery and all accompanying documentation.
6. Inspect the existing packages table to determine whether this is a new dataset concept or a new release of an existing dataset.

Rules:

- The registry dictionary is authoritative.
- Never alter files under data/**/raw/.
- Never silently modify registry schemas.
- Never invent metadata.
- Never invent IDs when a project script exists to mint them.
- Use project-relative POSIX paths in registry fields.
- Use legal vocabulary term_code values only.
- One party-role row represents one role assertion.
- Preserve uncertainty and conflicting evidence explicitly.
- Make the smallest set of changes necessary for this one registration.
- Do not "clean up" unrelated registry rows.

Workflow:

- A. Produce a written plan and identify whether this is a new dataset or release.
- B. Use scripts/mint_ids.py for IDs that genuinely need to be created.
- C. Create the standard data/<ds_iid>/<release_slug>/raw/ structure if needed, but do not move/delete existing source material without explicit instruction.
- D. Generate the manifest with the project manifest tool.
- E. Draft/update the packages row.
- F. Add parties only if they are not already registered.
- G. Add one role row per supported role assertion.
- H. Run the registry validator.
- I. Show me a diff and a review report before considering the registration complete.

Review report must include:

- evidence used;

- inferred values;
- unresolved fields;
- validation results;
- any proposed vocabulary or schema changes;
- anything you deliberately left untouched.

12.5 Review prompt

A second LLM pass can be useful if it is framed as adversarial review rather than rewriting.

Review this proposed registry change against the source materials and the registry dictionary.

Do NOT improve prose merely for style.

Look specifically for:

- unsupported claims;
- invented dates, citations, rights, identifiers, coordinates, or organizations;
- confusion between dataset identity and release identity;
- invalid null tokens;
- invalid vocabulary values;
- multivalue cells using the wrong delimiter;
- contributor roles that are inferred without evidence;
- duplicate parties;
- inconsistent source names;
- lineage claims that are not actually derivational;
- absolute local file paths;
- manifest fields that appear hand-entered rather than script-generated;
- fields whose value conflicts with source documentation.

Return:

1. errors that must be corrected;
2. uncertainties needing human judgment;
3. machine-validation issues;
4. fields that appear well supported;
5. a concise recommendation: accept, revise, or reject the draft.

12.6 Evidence discipline

A strong first-pass workflow should preserve a small evidence trail.

For important fields, the LLM should say where the value came from:

Registry field	Proposed value	Evidence	Confidence
source_org	...	source README	direct
published_date	...	repository page	direct
coverage_end	...	inferred from latest observation date	inferred
license	unk	no license found	unresolved

This is especially helpful during workshop registration because it turns review into verification rather than reconstruction.

12.7 What LLMs are good at here

LLMs are well suited to:

- reading heterogeneous documentation;
- consolidating repeated metadata;
- locating candidate citations;
- identifying names and organizations;
- summarizing methods;
- comparing a proposed row to the dictionary;
- spotting missing fields;
- generating review questions.

12.8 What requires caution

LLMs commonly overreach by:

- filling plausible dates;
- treating a website as a formal citation;
- assuming an open website implies an open license;
- inferring organizational affiliation from context;
- confusing data publication date with observation dates;
- treating any earlier release as direct lineage;
- selecting contributor roles from job titles rather than documented contributions.

The prompt should explicitly discourage these behaviors.

12.9 Final rule

An LLM can produce a useful first draft.

A human curator is responsible for deciding that a release is **registered**.

13 Validation and review

Registry quality depends on both machine validation and human review.

Neither is sufficient alone.

13.1 Machine validation

The validator should eventually check at least:

13.1.1 Structure

- exact header order against the dictionary;
- no missing or extra columns;
- UTF-8 encoding;
- expected CSV dialect;
- no duplicated primary identifiers.

13.1.2 Requiredness

- required fields populated;
- illegal null tokens rejected;
- conditional fields checked where machine rules are possible.

13.1.3 Patterns and types

- UUID validity;
- IID syntax;
- release-slug syntax;
- ISO date formatting;
- SHA-256 length/characters;
- integers/floats parsed correctly.

13.1.4 Controlled values

- enum values belong to the inline allowed list;
- vocabulary values exist as active or historically valid `term_code` values;
- registry table identifiers exist in `registry-table-iid.csv`.

13.1.5 Referential integrity

- party-role `dataset_id` exists;
- release-specific role `release_id` exists and belongs to the stated dataset;
- `party_id` exists;
- `broader_id`, `related_ids`, and `replaced_by` refer to valid concepts where applicable.

13.1.6 File integrity

- manifest path exists;
- recorded manifest path follows convention;
- manifest hash matches;
- optional full manifest verification passes;
- paths are project-relative, not absolute.

13.2 Conditional role validation

Party roles require richer validation because the legal `role_code` depends on `role_schema`.

Example:

```
role_schema=credit
```

should trigger validation against:

```
vocab/credit-roles.csv
```

and, if populated, contribution degree should be checked against:

```
vocab/credit-contribution-degree.csv
```

This is a planned enhancement to the generic validator.

13.3 Human review

Human review must consider things a validator cannot reliably determine.

13.3.1 Identity review

- Is this really a new dataset?
- Is the party already registered under another name?
- Does this release overlap but not descend from an earlier release?

13.3.2 Provenance review

- Is the source organization correct?
- Is the citation faithful to the source?
- Did a collaborator provide files that originated elsewhere?

13.3.3 Rights review

- Is the stated license actually attached to this dataset?
- Are there redistribution restrictions?
- Does an agency permission apply only to certain subsets?

13.3.4 Contributor review

- Is the CRediT role supported by evidence?
- Is the role dataset-level or release-specific?
- Are field collectors and curators being made visible rather than erased?

13.3.5 Scientific review

- Does the abstract accurately describe what is in the files?
- Are spatial and temporal coverage statements defensible?
- Is the processing summary technically accurate?

13.4 Suggested review checklist

Before changing `release_status` to a completed/registered state:

- ☐ Dataset vs. release decision reviewed.
- ☐ Required IDs minted by the project script.
- ☐ Raw acquisition preserved unchanged.
- ☐ Manifest generated.
- ☐ Required package fields reviewed.
- ☐ Source citation/provenance reviewed.
- ☐ Rights reviewed.
- ☐ Parties deduplicated.
- ☐ Party roles supported by evidence.
- ☐ Controlled values validated.
- ☐ Paths are portable/project-relative.
- ☐ Uncertainties documented.
- ☐ Validator passes.
- ☐ Another person or a deliberate second-pass review has inspected the entry.

13.5 Errors vs. warnings

Validators should distinguish:

13.5.1 Errors

Conditions that violate the registry standard.

Examples:

- missing required ID;
- invalid controlled value;
- broken foreign key;
- malformed release slug;
- header inconsistent with dictionary.

13.5.2 Warnings

Conditions that may be valid but deserve attention.

Examples:

- a release has no DOI;
- a rights field is `unk`;
- a party lacks ORCID;
- temporal coverage is absent because the dataset is timeless/reference-only;
- a release has no public landing page.

Do not turn every incomplete metadata field into an error; that encourages invented values.

14 Governance and versioning

The registry is a shared standard. It therefore needs rules for change.

14.1 Separate data changes from schema changes

Adding a new package row is a **data change**.

Adding a new column to `registry--packages.csv` is a **schema change**.

Those should not be treated as equivalent Git edits.

Schema changes require broader review because they affect every project using the standard.

14.2 Proposed change workflow

For a schema change:

1. describe the problem;
2. explain why existing fields/vocabularies cannot represent it;
3. propose the smallest change;
4. update the dictionary;
5. update the relevant physical CSV header;
6. update vocabularies;
7. update validation;
8. document migration requirements;
9. test against existing project registries;
10. merge only after review.

14.3 Preserve backward interpretability

Do not silently delete fields or vocabulary terms that have already been used.

If a concept is no longer preferred:

- deprecate it;
- keep its historical definition;
- point to a replacement when appropriate.

If a registry field must be renamed or removed, document the migration and consider a schema version boundary.

14.4 Versioning the registry standard

The lab should tag releases of the template repository.

A practical pattern could be:

```
v0.x    active design / workshop phase
v1.0    first stable registry standard
v1.x    backward-compatible additions/clarifications
v2.0    intentionally breaking schema changes
```

The exact versioning policy can be formalized later, but tagged states are valuable because project registries can say which template version they implement.

14.5 Git workflow

A lightweight shared workflow is sufficient:

- create a branch for non-trivial schema/vocab changes;
- make changes in small logical commits;
- run validation;
- open a pull request;
- review both the machine-readable change and documentation;
- merge after agreement.

Routine package registrations may use a simpler process depending on the project.

14.6 Vocabulary governance

A vocabulary change should distinguish:

14.6.1 New concept

Add a new concept with a stable ID and source.

14.6.2 Label refinement

If meaning is unchanged, update the preferred/alternate label and modification date carefully.

14.6.3 Meaning change

Do not reuse an existing code for a different concept. Create a new concept and deprecate the old one if necessary.

14.6.4 External standard update

Document which version/state of the external standard is being followed and preserve the source.

14.7 Roles and attribution

Contributor metadata can be sensitive in a social sense even when not legally sensitive.

Do not assign CRediT roles merely to reward or rank people. Record documented contributions. CRediT itself is a contribution taxonomy and is not intended to define authorship.

14.8 Auditability

Git history, manifests, stable identifiers, and source evidence together create an audit trail.

The goal is not bureaucracy. The goal is to ensure that a future researcher can reconstruct:

- what entered the project;
- when;
- from where;
- in what exact form;
- under what conditions;
- with whose contributions;
- and how the metadata evolved.

15 Scripts and automation

The registry standard is intentionally usable without a database server. Small command-line tools provide repeatable operations around CSV files and filesystem releases.

15.1 Core generic tools

15.1.1 `mint_ids.py`

Purpose:

- mint dataset IDs/IIDs;
- mint release IDs/slugs;
- mint party IDs/IIDs.

Typical use:

```
python scripts/mint_ids.py dataset kenai-wetlands
python scripts/mint_ids.py release kenai-wetlands <dataset-id> --ingest 2026-09-15 --rev 1
python scripts/mint_ids.py party jane-smith
```

IDs should be minted by the tool rather than typed manually.

15.1.2 `manifest.py`

Purpose:

- walk a frozen raw release;
- compute SHA-256 for each file;
- produce a deterministic manifest;
- report manifest hash, file count, and bytes.

The full production version should record project-relative paths and support verification.

15.1.3 validate_registry.py

Purpose:

- compare CSV headers to the dictionary;
- enforce required fields;
- validate enums and vocabularies;
- check foreign keys;
- enforce path and identifier conventions;
- eventually implement conditional role-schema validation.

Validation should be run before registry changes are merged.

15.2 Optional/source-specific tools

The AKSDB work produced several useful utilities that should be treated as optional tooling rather than mandatory parts of the registry standard.

15.2.1 inventory.py

Profiles fields in extracted CSVs and creates a machine-generated field inventory with type inference and quality flags.

Useful when a project must harmonize large heterogeneous tabular deliveries.

15.2.2 extract_mdb.py

Reproducibly extracts Microsoft Access `.mdb/.accdb` deliveries with `mbdtools`, capturing source tables and schema.

Useful only for projects that receive Access databases.

15.2.3 extract_props.py

Recovers Access field descriptions, properties, and query definitions that are not exposed by basic table export.

15.2.4 convert_map.py

Converts legacy wide mapping workbooks into a normalized long-format mapping table.

15.3 Core vs. optional philosophy

The lab standard should avoid requiring every project to inherit the full complexity of the most complicated project.

A good boundary is:

```
core:
  identity
  packages
  parties
  roles
  vocabularies
  manifests
  validation

optional:
  field inventories
  database extraction
  source-schema capture
  harmonization maps
  scientific schema validation
```

15.4 Running scripts from anywhere

A mature template should include shared path-resolution logic so tools can locate the project root regardless of the current working directory.

Registry paths should always be written relative to the project root.

15.5 Reproducibility

If a manual transformation is repeated more than once, strongly consider turning it into a script.

Examples:

- extracting tables;
- renaming columns;
- fixing encodings;
- creating manifests;

- generating inventories;
- validating vocabularies;
- compiling reports.

The objective is not to automate every judgment. It is to automate repeatable operations so human attention can focus on judgment.

16 Worked example

This example demonstrates the full reasoning process using a fictional dataset.

16.1 Scenario

A collaborator sends a folder:

```
peat-depth-survey-2026/  
  peat_depths.csv  
  site_notes.docx  
  README.txt
```

The README says:

- the data are from a 2026 Kenai Peninsula field campaign;
- Jane Smith led field data collection;
- Alex Lee developed the sampling design;
- the files are an updated delivery replacing a preliminary folder sent two months earlier;
- no DOI exists;
- redistribution is permitted within the collaborative project, but public release has not yet been approved.

A preliminary release is already registered under:

```
dataset_id = 3e...  
ds_iid     = kenai-peat-depths
```

16.2 Step 1: dataset or release?

This is **not** a new dataset concept.

It is an updated release of `kenai-peat-depths`.

Reuse:

```
dataset_id = 3e...
ds_iid      = kenai-peat-depths
```

Mint a new release.

16.3 Step 2: mint the release

```
python scripts/mint_ids.py release    kenai-peat-depths    3e...    --ingest 2026-09-15    --rev
```

Suppose the tool returns:

```
release_id    = <uuidv7>
release_slug   = kenai-peat-depths--ing20260915--r002
```

16.4 Step 3: preserve raw files

Create:

```
data/kenai-peat-depths/kenai-peat-depths--ing20260915--r002/raw/
```

Copy the three supplied files into `raw/` unchanged.

16.5 Step 4: generate manifest

Run:

```
python scripts/manifest.py    data/kenai-peat-depths/kenai-peat-depths--ing20260915--r002/raw
```

Record the generated:

- manifest path;
- manifest SHA-256;
- file count;
- total bytes.

16.6 Step 5: package metadata

Potential values include:

```
title:
Kenai Peninsula peat-depth field survey, 2026 updated delivery

purpose:
Updated field delivery for peat-depth modeling and targeted sampling analysis.

source_org:
<collaborator organization, if documented>

doi:
nap

coverage_start:
2026-...

coverage_end:
2026-...

intellectual_rights:
Collaborative-project redistribution permitted; public release not yet approved.

processing_summary:
Updated delivery superseding the preliminary 2026 handoff; no local transformations applied t

release_status:
<appropriate current workflow term>
```

Notice that the rights statement is preserved as a condition rather than incorrectly converting it into an open license.

16.7 Step 6: parties

Search the existing parties table for Jane Smith and Alex Lee.

If absent, create new party IDs/IIDs.

Do not add job titles as permanent roles on their identity records.

16.8 Step 7: roles

If using CRediT:

Jane's field contribution supports:

```
role_schema = credit
role_code   = investigation
```

Alex's sampling-design contribution may support:

```
role_schema = credit
role_code   = methodology
```

If the README explicitly indicates Jane led the investigation role and multiple investigators exist:

```
contribution_degree = lead
```

If that ranking is not stated or needed, leave degree unassigned.

Create one row per role assertion.

16.9 Step 8: role evidence

A defensible record might use:

```
role_basis   = source-declared
role_source  = data/kenai-peat-depths/.../raw/README.txt
```

This is preferable to assigning roles solely because a person's job title sounds compatible with a CRediT role.

16.10 Step 9: review uncertainties

Questions for human review might include:

- Does the updated delivery fully supersede the preliminary release?
- What exact temporal coverage dates are present in the CSV?
- Should the dataset-level title remain stable across releases?
- Is the collaborator organization explicitly documented?
- Should rights be encoded as `license=nap` plus a detailed `intellectual_rights` statement?
- Is the preliminary release a direct upstream ancestor or merely an earlier snapshot?

These questions should be resolved explicitly rather than guessed.

16.11 Step 10: validate and commit

Run validation, inspect the diff, and commit the registration.

The result is a release that can later be transformed and harmonized without losing the context of what originally entered the project.

17 Template readiness and remaining implementation work

This chapter tracks the gap between the intended registry standard documented in this book and the current files that have evolved from AKSDB/JLDR prototypes.

It should be reviewed before the template repository is tagged as a stable release.

17.1 Architecture decisions already made

The intended standard is now:

- no numeric prefixes on top-level directories;
- **registry/**, **vocab/**, **data/**, **scripts/**, **docs/**;
- glossary moved to a separate repository;
- five canonical registry CSV artifacts;
- one common vocabulary schema;
- dictionary as the field-level specification for everything in **registry/**.

17.2 Reported/created changes

These have been discussed or created during the registry design process:

- ☒ Remove numeric prefixes from registry/vocab directories.
- ☒ Rename **table-iid.csv** to **registry-table-iid.csv**.
- ☒ Create **registry--vocab-schema.csv** as a header-only schema file.
- ☒ Define a rich common vocabulary structure.
- ☒ Create draft CRediT role vocabulary.
- ☒ Create draft CRediT contribution-degree vocabulary.
- ☒ Decide to maintain the glossary in a separate repository.
- ☒ Decide that one party-role row should represent one role assertion.

Note

Items marked complete above reflect design decisions and generated artifacts from the workshop preparation. Confirm that the corresponding files have actually been committed to the local repository before treating them as implemented.

17.3 Registry/dictionary updates still needed

- ☐ Merge the 16 vocabulary-schema dictionary rows into the canonical `registry--dictionary.csv`.
- ☐ Change their `table_iid` from legacy `vocab` to `vocab-schema`.
- ☐ Update the dictionary's `table_iid` vocab reference to `registry-table-iid.csv`.
- ☐ Ensure `registry-table-iid.csv` contains exactly the canonical registry artifact identifiers:
 - `packages`
 - `parties`
 - `party-roles`
 - `dictionary`
 - `vocab-schema`
- ☐ Normalize `null_values` / `null_tokens` to the chosen canonical field name `null_tokens`.

17.4 Identifier naming cleanup still needed

The prototype files contain naming drift. The intended v1 convention is:

```
ds_id_uuidv4      -> dataset_id
release_id_uuidv7 -> release_id
party_uuidv4       -> party_id
party_slug/party_id -> party_iid where the value is a readable IID
eml_packageId      -> eml_package_id
```

Before v1:

- ☐ Update physical CSV headers.
- ☐ Update dictionary rows.
- ☐ Update scripts.
- ☐ Migrate existing registry records deliberately.
- ☐ Update validation.
- ☐ Document the schema migration.

17.5 Parties updates still needed

- ☐ Remove `role_default` from `registry--parties.csv`.
- ☐ Remove `role_default` from the dictionary.
- ☐ Confirm person/organization requiredness rules.
- ☐ Confirm privacy guidance for email fields.

17.6 Party-role redesign still needed

Replace the legacy role table with the target schema:

```
role_assignment_id
dataset_id
release_id
party_id
role_schema
role_code
contribution_degree
role_basis
role_source
notes
```

Then:

- ☐ Update dictionary rows.
- ☐ Migrate any existing role rows.
- ☐ Add UUID minting for `role_assignment_id`.
- ☐ Update validator foreign-key checks.
- ☐ Implement dataset-level vs. release-level role semantics.
- ☐ Decide and create `role-basis.csv` if adopted.

17.7 CRediT integration still needed

- ☐ Add `credit-roles.csv` to `vocab/`.
- ☐ Add `credit-contribution-degree.csv` to `vocab/`.
- ☐ Ensure `role-schema.csv` contains `credit`.
- ☐ Link the role schema to the correct role vocabulary in `validation/configuration`.
- ☐ Validate `role_code` conditionally.
- ☐ Validate degree only when appropriate.
- ☐ Preserve canonical CRediT labels/definitions and source URLs.

- ☐ Confirm whether the registry will support DataCite and EML role vocabularies immediately or later.

17.8 File naming cleanup

The final convention should be applied consistently.

Recommended:

```
registry/  
  registry--packages.csv  
  registry--parties.csv  
  registry--party-roles.csv  
  registry--dictionary.csv  
  registry--vocab-schema.csv  
  
vocab/  
  registry-table-iid.csv  
  content-domains.csv  
  release-status.csv  
  role-schema.csv  
  credit-roles.csv  
  credit-contribution-degree.csv
```

- ☐ Remove legacy `vocab--` filename prefixes if that is the final decision.
- ☐ Update every `vocab_ref` in the dictionary accordingly.
- ☐ Update scripts that assume old filenames.

17.9 Script updates

- ☐ Remove old numbered-directory assumptions.
- ☐ Update path-resolution code to the final tree.
- ☐ Update `mint_ids.py` field names.
- ☐ Update `validate_registry.py` for the final headers.
- ☐ Add validation for all vocab files against `registry--vocab-schema.csv`.
- ☐ Add conditional role-vocabulary validation.
- ☐ Ensure manifest paths are emitted project-relative rather than absolute.
- ☐ Add manifest verification mode if the lightweight template script does not yet have it.

17.10 Documentation synchronization

Once the CSVs are migrated:

- ☐ Compare every documentation field table against the actual dictionary.
- ☐ Run a documentation link check.
- ☐ Render the Quarto book.
- ☐ Tag the registry template version used by the documentation.

17.11 Definition of “ready for lab use”

The template is ready for broader lab adoption when:

1. the five registry artifacts exist with final headers;
2. the dictionary defines every one of those headers;
3. all core vocabularies conform to `registry--vocab-schema.csv`;
4. CRediT roles are wired into party-role validation;
5. the generic validator passes on a clean template;
6. at least one real dataset has been registered end to end;
7. another lab member can complete a registration using the documentation without relying on undocumented oral instructions.